고급모바일프로그래밍

# 텍스트 기반 인생게임
# 11주차 구현 기술 확정 보고서

프로젝트 제안서 · 요구사항 분석서 · UI 설계 방향 · 구현 기술 통합본

| | |
|---|---|
| 팀명 | 인생게임 제작 |
| 대표 제출자 | 김선한 (2271179) |
| 팀원 | 김재겸 · 김우영 · 신기훈 · 정원률 |
| 작성 기준일 | 2026년 11월 12일 |

# 1. 보고서 목적과 현재 프로젝트 기준

이 보고서는 기존 프로젝트 제안서·요구사항 분석서·UI 설계 방향을 현재 소스 코드와 콘텐츠 기준으로 다시 묶고, 11주차 과제인 "구현 기술 그룹핑 및 사용 목적·선정 이유 정리"를 반영한 최종 기술 확정 문서다. 설계 문구는 코드와 JSON에서 확인되는 범위만 사용했으며, 향후 계획과 현재 구현을 구분했다.

## 현재 구현 규모

| 항목 | 현재 기준 | 근거 |
|---|---|---|
| 플랫폼 | Android 네이티브 앱 | app/build.gradle.kts, AndroidManifest |
| 사건 / 선택지 | 352개 / 1,009개 | events.sample.json |
| 결말 | 66개 | endings.sample.json |
| 이미지 카탈로그 | 524개 항목, 382개 라우팅 자산 | image_ids.sample.json, image_routing.json |

# 2. 구현 기술 그룹핑

| 기술 그룹 | 확정 기술 | 사용 목적 | 선정 이유 |
|---|---|---|---|
| 모바일 플랫폼 | Android 네이티브, Kotlin, Android SDK 35, minSdk 26 | Android 앱 실행, 생명주기, 파일·권한·에뮬레이터 연동 | 현재 과제가 Android 앱이며 Kotlin 코드와 Compose 테스트 자산이 이미 축적되어 플랫폼별 최적화와 검증이 쉽다. |
| UI/화면 | Jetpack Compose, Material 3, Navigation Compose | 시작·캐릭터 설정·게임 진행·결말 화면과 상태 기반 전환 구현 | 선언형 UI로 사건·선택지·결과 상태를 즉시 다시 그릴 수 있고, 작은 화면·접근성 테스트와 결합하기 쉽다. |
| 상태 관리 | ViewModel, lifecycle-runtime-compose | 현재 생명 진행, 사건, 선택 결과, 이미지 선노출, 컷신 상태 유지 | 화면 재구성에도 게임 세션을 보존하고 UI와 순수 게임 로직을 분리한다. |
| 게임 도메인 로직 | 순수 Kotlin GameEngine·GameSession·도메인 모델 | 스탯 변화, 조건부 사건, 결과 확률, 플래그, 사망·결말 판정 | 서버 없이도 결정적이고 재현 가능한 싱글 플레이를 제공하며 단위 테스트가 쉽다. |
| 콘텐츠 저장 | JSON assets, org.json, V4ContentParser | 사건·선택지·초점·결말·인생 사실·이미지 메타데이터 로드 | 코드 수정 없이 콘텐츠를 확장하고 오프라인 앱에 함께 패키징할 수 있다. |
| 이미지 처리 | PNG 도트풍 자산, BitmapFactory, PixelAssetLoader, LruCache | 사건별 장면 이미지와 성장 캐릭터를 안전하게 디코드·리사이즈·캐시 | 저사양 기기 메모리 사용을 제한하면서 352개 사건의 이미지 라우팅을 명시적으로 유지한다. |
| 사용자 사진/AI 보조 | Photo Picker/ContentResolver, 선택적 OpenAI 이미지 API, 로컬 Fake 응답 | 사용자 사진 동의·권한·실패 상태를 분리하고 얼굴 시트 생성 선택 기능 제공 | API 키가 없어도 기본 흐름이 동작하도록 로컬 대체 경로를 둔다. 키는 local.properties로만 주입한다. |
| 영속 저장 | SharedPreferences 기반 ProgressStore, 버전 마이그레이션 | 진행 상태·스탯·플래그·선택 기록·장면 상태 저장·복원 | 작은 단일 플레이 저장에는 의존성이 적고, 구버전 저장 v1~v3 호환을 유지하기 쉽다. |
| 품질 검증·빌드 | Gradle, JUnit, Compose UI Test, Python 콘텐츠 검증 | 게임 로직·화면·JSON 참조·이미지 라우팅·APK 패키징 검증 | 콘텐츠와 코드가 함께 바뀌는 프로젝트라 자동 검증을 병합 기준으로 사용한다. |

# 3. 강의 내용과 프로젝트 기술 선택의 관계

11번 강의노트는 백엔드 서비스를 데이터 관리·비즈니스 로직·API·상태 관리로 설명하고, 10번 강의노트는 네이티브·웹·하이브리드·크로스 플랫폼을 비교한다. 우리 프로젝트는 인터넷 기반 다중 사용자 서비스가 아니라 오프라인 싱글 플레이 게임이므로, 서버 REST/GraphQL/gRPC/WebSocket을 필수 기술로 채택하지 않았다.

## 백엔드 서비스 개념을 프로젝트에 적용한 방식

| 강의 개념 | 프로젝트 대응 | 적용 범위 |
|---|---|---|
| 데이터 관리 | ContentRepository + JSON assets | 사건·선택지·결말·이미지 ID CRUD 성격의 읽기 및 검증 |
| 비즈니스 로직 | GameEngine / GameSession | 스탯 조건, 결과 확률, 플래그, 사망·결말 판정 |
| 상태 관리 | ViewModel + ProgressStore | 현재 사건, 세션, 저장·복원, 컷신·이미지 상태 |
| API 제공 | 외부 서버 API는 기본 경로에서 사용하지 않음 | 오프라인 플레이 우선. 선택적 사진 변환 API만 별도 실패 상태로 격리 |
| 보안 | local.properties의 키 주입 | OpenAI 키를 소스와 저장소에 포함하지 않음 |

## 크로스 플랫폼 선택 비교

| 선택지 | 장점 | 이번 프로젝트 판단 |
|---|---|---|
| Android 네이티브 Kotlin | Android UI·생명주기·권한·에뮬레이터 검증이 직접적 | 채택. 과제 플랫폼과 현재 코드가 Android에 고정되어 있고, Compose 테스트 및 APK 시연이 준비되어 있다. |
| Flutter | 단일 코드베이스로 Android/iOS/웹 확장 가능 | 이번 제출 범위에서는 기존 Kotlin·Compose 자산을 다시 작성해야 하므로 채택하지 않음 |
| React Native | JavaScript/React 기반의 다중 플랫폼 개발 | 현재 프로젝트에 JS 번들·브리지 구조가 없어 전환 비용이 큼 |
| 웹/하이브리드 | 배포와 공통 코드 측면의 장점 | 오프라인 Android APK 시연과 도트 이미지 캐시 요구에 맞지 않음 |

# 4. 시스템 구성과 데이터 흐름

사용자 입력은 Compose 화면에서 ViewModel로 전달되고, ViewModel은 GameSession/GameEngine에 선택을 위임한다. 엔진은 JSON으로 읽은 사건·조건·결말을 기준으로 새 상태를 만들고, ProgressStore가 진행 상태를 SharedPreferences에 기록한다. 화면은 상태 변경을 관찰해 사건 이미지, 선택지, 결과 애니메이션, 인생 요약을 다시 그린다.

## 구성 요소별 책임

| 계층 | 주요 구성 | 책임 |
|---|---|---|
| Presentation | LifeGameApp, screens, components, Compose | 사용자 입력, 화면 렌더링, 타이핑·컷신·장면 보기·결말 표시 |
| State | LifeGameViewModel, UI state | 세션 상태와 화면 상태 연결, 저장 복원 후 화면 복구 |
| Domain | GameEngine, GameSession, GameModels | 규칙·조건·확률·스탯·결말의 순수 계산 |
| Data | ContentRepository, V4ContentParser, ProgressStore | JSON 파싱, 저장 문자열 직렬화, 버전 호환 |
| Assets | events/endings/focuses/image_routing JSON + images | 실제 게임 콘텐츠와 사건별 이미지 파일 제공 |

# 5. 요구사항과 기술 매핑

| 요구사항 영역 | 현재 기술 구현 | 검증 방법 |
|---|---|---|
| 새 게임·캐릭터 설정 | Compose 화면 + ViewModel 초기화 | Compose UI 테스트, 에뮬레이터 시연 |
| 사건·선택·결과 | JSON + GameEngine + DecisionScreen | JUnit 도메인 테스트, 콘텐츠 검증 |
| 시기별 초점·인생 사실 | V4ContentParser + LifeFacts + flags | 집중 스탯별 경로/결말 테스트 |
| 이미지 선노출·라우팅 | ImageRouting + PixelAssetLoader + SceneImage | 382개 라우팅 검증, 실제 APK 설치 시연 |
| 저장·복원 | ProgressStore + 저장 버전 호환 | 재실행·복원 흐름 수동/단위 검증 |
| 결말·인생 요약 | EndingTier + LifeFacts summary | 시기·원인·교육·직업·가족·업적 표시 확인 |

# 6. 구현 현황과 검증 기준

현재 코드와 최신 통합본에서 확인한 기술 구현 현황이다. "기술을 선택했다"는 사실과 "모든 콘텐츠 검수가 끝났다"는 상태를 구분한다.

| 검증 항목 | 결과 | 비고 |
|---|---|---|
| Debug/Release 단위 테스트 | 통과 | 게임 로직·저장·이미지 라우팅 테스트 실행 |
| assembleDebug | 성공 | 에뮬레이터 설치용 APK 생성 |
| 콘텐츠 JSON 검증 | 통과 | 사건 352개, 선택 결과 1,009개, 결말 66개 기준 |
| 이미지 라우팅 | 382개 라우팅 자산 | 사건 이미지 매핑은 352/352로 연결 |
| 오프라인 기본 흐름 | 구현 | 외부 API 없이 게임 진행 가능 |
| 외부 사진 변환 | 선택 기능 | 키·권한·실패 상태를 분리하고 기본 Fake 경로 유지 |

# 7. 제한 사항과 관리 기준

첫째, 본 앱의 핵심 실행 경로는 Android 로컬 콘텐츠와 로컬 게임 엔진이다. 따라서 REST API, GraphQL, gRPC, WebSocket 서버를 구현했다고 기술하지 않는다. 둘째, 이미지 라우팅 연결은 "파일을 사건에 연결하는 기술 상태"이고, 이미지 승인·의미 검수 상태와 동일하지 않다. 셋째, OpenAI 사진 변환은 API 키가 있을 때만 선택적으로 실행되며, 키가 없거나 네트워크가 없으면 기본 캐릭터와 로컬 Fake 응답으로 게임을 계속할 수 있다.

## 병합 기준

팀원 브랜치 병합 전에는 Gradle 테스트, 콘텐츠 검증, 이미지 파일·라우팅 검증, 에뮬레이터 시연 체크리스트를 함께 확인한다. 변경 사항은 PR과 커밋으로 남기며, 보고서에는 실제 구현과 계획을 분리해 기록한다.

# 8. 결론

현재 프로젝트의 확정 구현 기술은 Android 네이티브 Kotlin과 Jetpack Compose를 중심으로, ViewModel 기반 상태 관리, 순수 Kotlin 게임 엔진, JSON assets 콘텐츠 관리, SharedPreferences 저장, Bitmap 이미지 로더·라우팅, Gradle/JUnit/Compose/Python 검증 체계로 구성된다. 이 조합은 강의에서 설명한 백엔드의 데이터 관리·비즈니스 로직·상태 관리 역할을 앱 내부 구조로 구현하면서, Android 과제의 오프라인 싱글 플레이 요구와 현재 팀의 코드 자산을 가장 적은 전환 비용으로 충족한다.

# 참고 자료

| 자료 | 활용 내용 |
|---|---|
| 10_크로스 플랫폼 개요.pptx | 네이티브·웹·하이브리드·크로스 플랫폼 비교 기준 |
| 11_백엔드 서비스 개요.pptx | 백엔드 서비스의 데이터 관리·비즈니스 로직·상태 관리·API 개념 |
| 6주차 프로젝트 제안서 및 요구사항 분석서 최종본 | 프로젝트 목표, 요구사항 ID, 데이터·화면·테스트 범위 |
| docs/week8_frontend_report.md, docs/week8_backend_submission.md | 현재 UI·인생 사실·게임 로직 구현 및 검증 결과 |
| app/build.gradle.kts 및 app/src/main | 실제 사용 라이브러리·구조·저장·이미지 처리 확인 |